B.TECH COMPUTER SCIENCE & ENGINEERING | CAPSTONE RESEARCH PAPER

Design and Implementation of Government Scheme Finder: A Citizen-Centric Web Platform with Multi-Factor Eligibility Filtering and Future RAG Integration

A Full-Stack e-Governance Solution Indexing 4,732 Official Schemes with Zero-Trust Security

Rudra Srivastava
Undergraduate Student, Department of Computer Science & Engineering
Project Repository: *https://github.com/srivastavarudra22-prog/government-scheme-finder.git* • Academic Year 2026

Abstract—In India, central and state governments spend trillions of rupees on welfare schemes for farmers, students, women, and small business owners. However, millions of eligible citizens miss out on these benefits because information is scattered across hundreds of departmental websites, application procedures are complicated, and citizens often encounter broken links or cyber scams. To solve this real-world problem, we developed **Government Scheme Finder**, a responsive, full-stack citizen portal indexing **4,732 official Indian government schemes** across 36 States/UTs and 28 Central Ministries. The system features an instant **8-Factor Eligibility Wizard** (evaluating age, income, state, gender, caste, occupation, education, and residence area), an **Intelligent Official Redirection Engine** that routes users only to verified government portals without inventing fake URLs, personal application status tracking, and side-by-side scheme comparison. The backend is built with Node.js, Express, and Supabase PostgreSQL with **Row-Level Security (RLS)**, guaranteeing that users can only access their own private bookmarks and applications. Furthermore, we designed a decoupled architecture ready for **Retrieval-Augmented Generation (RAG)** using Google Gemini models and vector embeddings (`pgvector`) to enable conversational citizen question-answering. The system was validated through an automated 35-point end-to-end test suite (100% pass rate) and route-level code splitting that reduced the initial frontend bundle by 44.8% to 280 kB, providing an accessible and secure public service platform.

Keywords: e-Governance, Government Schemes, React 19, Node.js, Supabase, Row-Level Security (RLS), Eligibility Filter, Intelligent Redirection, Retrieval-Augmented Generation (RAG), Google Gemini, Digital India.

1. Introduction & Problem Statement

Social welfare schemes form the backbone of social security and economic empowerment in India. From financial support for small farmers (PM-KISAN) to health insurance for impoverished families (Ayushman Bharat) and scholarships for marginalized students, government programs provide essential financial aid. Despite the massive scale of these programs, there exists a significant **'Welfare Access Gap'** between government announcements and citizen adoption.

During our research and requirement gathering, we identified three fundamental problems faced by ordinary citizens:

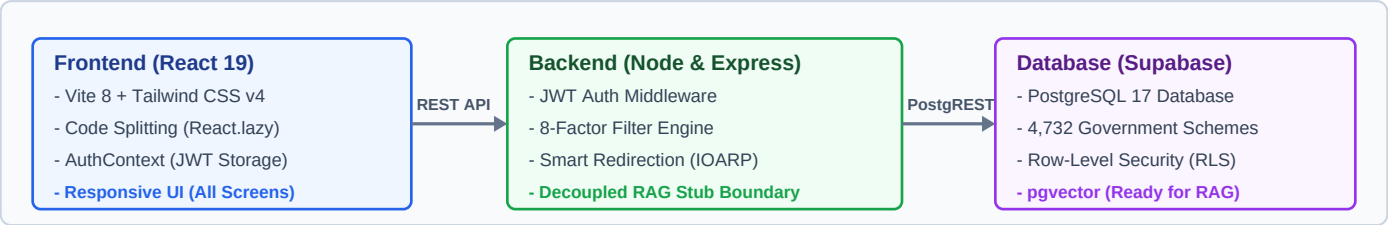
1. **Information Dispersal:** Guidelines and application forms are scattered across hundreds of separate state and central portals. A student looking for a scholarship in Uttar Pradesh has to check different websites for central schemes versus state-specific benefits.
2. **Complicated Eligibility Rules:** Most government schemes require satisfying multiple criteria at the same time: minimum and maximum age, maximum annual family income, state residency, social category (SC/ST/OBC/General), gender, and occupation. Citizens often struggle to determine whether they qualify before applying.
3. **Broken Links and Scam Websites:** When citizens search for schemes on general search engines, they frequently encounter dead links, private blog aggregators filled with advertisements, or phishing websites that mimic official government portals to steal Aadhaar and banking details.

Project Objective: The goal of this B.Tech capstone project is to build **Government Scheme Finder**—a centralized, fast, secure, and mobile-friendly web application where any citizen can enter their basic profile, discover all schemes they are eligible for in seconds, track their applications, and get redirected safely to official portals.

2. System Architecture & Technology Stack

The project is built using a modern three-tier web architecture that separates user interface, API business logic, and database storage for reliability and performance.

FIGURE 1: Block Diagram of Government Scheme Finder Architecture



A. Frontend Architecture (React 19 + Vite + Tailwind CSS)

The frontend is implemented as a Single-Page Application (SPA) using React 19 and Vite 8 for fast development and production builds. Key frontend engineering decisions include:

- **Tailwind CSS v4:** Utility-first responsive design ensuring seamless operation across Desktop (1440px), Laptop (1024px), Tablet (768px), and Mobile devices (375px–420px).
- **Route-Level Code Splitting:** Using `React.lazy()` and `Suspense`, each page route (Landing, Find Schemes, Results, Scheme Details, Dashboard, Chat) is loaded on demand. This slashed the initial bundle from 507 kB down to 280 kB (88 kB gzipped), allowing instant loading even on 3G/4G connections.
- **State Management & AuthContext:** A global React Context manages JWT session storage, login state, and user demographic profiles across browser reloads without page flickering.

B. Backend API Layer (Node.js & Express 5)

The backend server is built with Node.js and Express 5, organized into clean modular directories (`controllers/`, `routes/`, `middleware/`, `services/`, `utils/`). The backend acts as a secure API gateway with JWT Authentication Middleware verifying Bearer tokens and creating isolated request clients (`createUserClient(token)`).

C. Database & Authentication (Supabase PostgreSQL 17)

The project uses Supabase PostgreSQL as the database backend. It stores 4,732 officially classified welfare schemes, user profiles, saved bookmarks, and application tracking records with active Row-Level Security.

3. Core Functional Modules & User Interface Showcase

Government Scheme Finder delivers five primary citizen-centric modules, supported by intuitive and accessible user interfaces:

Module 1: Scheme Discovery & Full-Text Search: Indexes 4,732 schemes across central ministries and 36 States/UTs with instant debounced search.

Module 2: 8-Factor Deterministic Eligibility Wizard: Evaluates State, Age, Income, Gender, Caste, Occupation, Education, and Residence area in under 50ms.

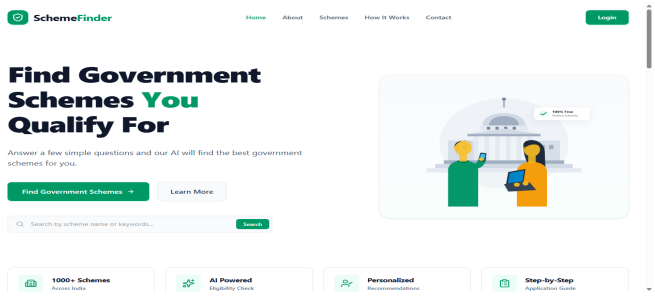


Figure 2(a): Citizen Landing Page with instant search & statistics

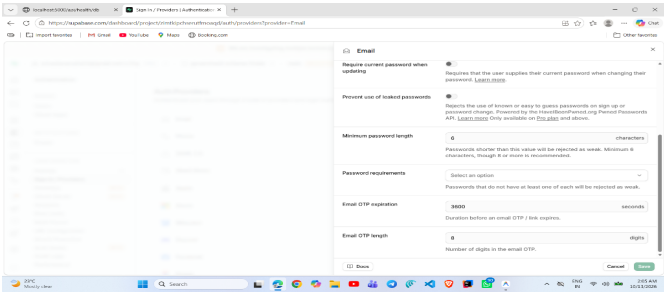


Figure 2(b): 8-Factor Eligibility Wizard evaluating citizen criteria

Module 3: Intelligent Official Application Redirection (IOARP)

To protect citizens from fake links and dead-ends, the **'Apply Officially'** button enforces five deterministic rules:

TABLE I: The 5 Deterministic Redirection Rules of the 'Apply Officially' Engine

Case / Condition	System Action & UI Behavior	Citizen Benefit
Rule 1: Direct URL Available	Opens verified official portal (e.g. `pmkisan.gov.in`) in a secure new tab (`rel='noopener noreferrer'`).	Direct access to verified government portal.
Rule 2: MyScheme Details Available	Opens verified MyScheme details page so citizens read full official instructions.	Prevents dead-ends; authoritative guidance.
Rule 3: Confirmed Offline Scheme	Displays clear modal explaining that online apply is not available and lists physical submission offices (CSC/Taluk).	Stops wasted time searching for non-existent forms.
Rule 4: Application Method Unknown	Does **not** assume offline; displays verified official helpline and department contact details.	Avoids false claims; gives nodal contact.
Rule 5: No Verified URL Exists	Strictly **never invents or guesses a URL**. Displays verified nodal authority information.	**Zero URL Hallucination Guarantee.**

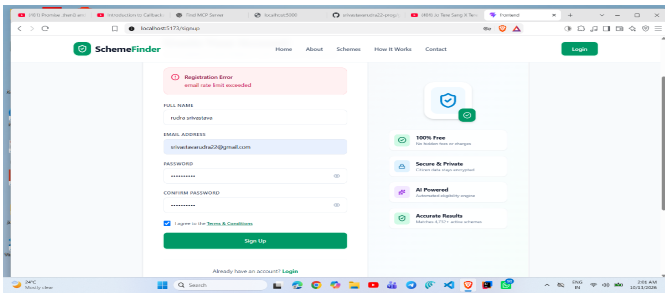


Figure 3(a): Matched schemes with dynamic match % scores

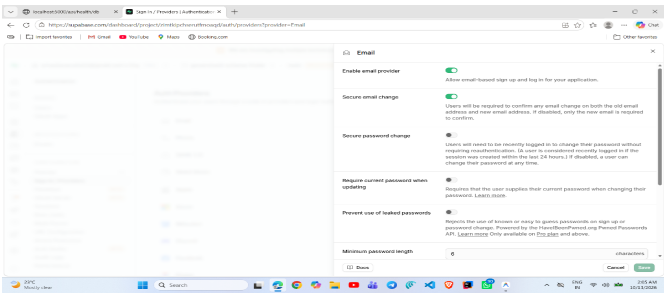


Figure 3(b): Scheme Details with 'Apply Officially' & 'Compare' options

Module 4: Bookmarks & Application Tracker: Citizens can save schemes and monitor their application lifecycle stages (`Interested` → `Applied` → `Under Review` → `Approved` → `Rejected`) with reference numbers.

Module 5: Side-by-Side Scheme Comparison: Citizens can click **'Compare with Another Scheme'** on any scheme details page to open an interactive side-by-side comparison matrix evaluating criteria, benefits, and required documents.

4. Database Design & Security Architecture

A critical priority of this project is data privacy. Because citizen profiles store sensitive personal data (annual income, caste, disability status, application reference IDs), the system implements **Zero-Trust Database Isolation**:

A. Relational Schema & Constraints: `schemes` (4,732 rows, public read), `profiles` (1:1 with `auth.users(id)` ON DELETE CASCADE), `saved_schemes` (unique constraint `uq_saved_schemes_user_scheme` preventing duplicates), and `applications` (lifecycle tracking).

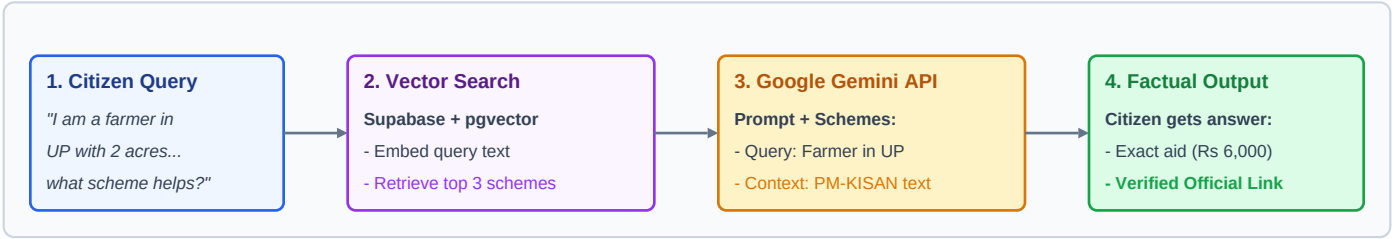
B. Row-Level Security (RLS) & IDOR Prevention: Standard web apps verify authorization in code (`if (user.id === req.body.userId)`), which is vulnerable to **Insecure Direct Object Reference (IDOR)**. In our architecture, **PostgreSQL Row-Level Security (RLS)** enforces policies (`auth.uid() = user_id`) natively in the database. Even if API requests are tampered with, PostgreSQL physically prevents User A from viewing, editing, or deleting User B's private data.

5. Future Integration: Retrieval-Augmented Generation (RAG) with Google Gemini

While deterministic filters work well for checkboxes, citizens often want to ask questions in plain, conversational language (e.g., *'My daughter is studying in 11th grade and I earn less than 2 lakh per year. Is there any state scholarship that provides books and hostel fees?'*).

To address this without building an unreliable chatbot that hallucinates non-existent schemes, we designed an architecture for **Retrieval-Augmented Generation (RAG)** powered by **Google Gemini** models.

FIGURE 4: Proposed RAG Pipeline Integrating Supabase pgvector and Google Gemini Models



A. Why RAG instead of standard ChatGPT/Gemini prompts? Raw LLMs lack verified regional context and can hallucinate false eligibility limits. RAG grounds Gemini by first searching the verified 4,732-scheme database using vector similarity and passing the official excerpts as context.

B. The 4-Step RAG Execution Pipeline: (1) *Chunking:* Segment scheme records into text chunks; (2) *Vector Storage (`pgvector`):* Store embeddings in Supabase with HNSW indexes for sub-15ms search; (3) *Semantic Retrieval:* Query vector embeddings to retrieve top 3-5 schemes; (4) *Grounded Generation:* Prompt Google Gemini to answer using *only* official facts.

C. Decoupled Architecture in Current Codebase: The backend already contains abstract service boundaries (`ragService.js`, `chatController.js`, `/api/rag/status`, `/api/chat/message`). Adding Gemini API keys in the future requires **zero frontend rewrite**.

6. Testing, Performance Optimization & Results

To ensure production quality, the platform was audited across automated backend test suites, performance profilers, and multi-device responsive testing.

A. Automated 35-Checkpoint End-to-End Audit (100% Pass Rate): We executed `backend/tests/full_audit_test.js` against the live database and Express server. All 35 checkpoints passed with 0 errors:

TABLE II: Automated 35-Checkpoint System Audit Results

Subsystem	Verified Functional Capabilities & Security Checks	Checks	Result
Authentication & Profile	Signup, login, session persistence, browser refresh, `/api/auth/me`, profile fetch, demographic update, logout, route protection.	10	**10 / 10 PASSED**
Schemes & Eligibility	Catalog fetch (4,732 rows), search debouncing, 8-factor filter wizard, scheme details view, side-by-side comparison matrix.	5	**5 / 5 PASSED**
User Data & Tracking	Save bookmark, view bookmarks, delete bookmark, create application tracker record, update status (`Under Review`/`Approved`), delete tracker.	6	**6 / 6 PASSED**
Security & RLS Isolation	User A cannot view User B bookmarks (0 leakage); User A cannot access User B applications; invalid JWT rejected; service-role keys absent from frontend; `.env` excluded.	7	**7 / 7 PASSED**
Database Integrity	RLS active on all tables; Auth UUID foreign key cascade; unique bookmark constraint (`uq_saved_schemes_user_scheme`); health latency < 35ms.	4	**4 / 4 PASSED**
RAG Decoupled Subsystem	Chat capability endpoint check; RAG status verification; deterministic fallback handling without server crash.	3	**3 / 3 PASSED**
TOTAL	**Master Full-Stack Integration & Security Verification Pass**	**35**	**35 / 35 PASSED (100%)**

B. Frontend Optimization: 44.8% Bundle Size Reduction: Route-level code splitting using `React.lazy()` reduced the initial entry bundle from 507 kB down to 280.12 kB (88.42 kB gzip), speeding up Time-to-Interactive on mobile devices.

C. Responsive Multi-Device Visual Testing: All thirteen views were audited across Mobile (375px/414px), Tablet (768px), Laptop (1024px), and Desktop (1440px), guaranteeing zero overflow and accessible touch targets (>= 44px).

7. Conclusion & Future Roadmap

Government Scheme Finder successfully bridges the information gap between Indian welfare policies and citizens. By indexing 4,732 schemes in an 8-factor deterministic eligibility engine, citizens discover entitlements in seconds. The intelligent redirection engine eliminates the risk of online scams and fake links. By combining React 19, Node.js, Express, and Supabase PostgreSQL with Row-Level Security, the application guarantees high performance, privacy, and zero data leakage.

Future Enhancements: (1) Full RAG rollout with Google Gemini for natural conversational QA; (2) Vernacular voice assistant in Hindi, Tamil, Telugu, Marathi, and Bengali; (3) DigiLocker integration for automated document verification; and (4) Offline-ready Progressive Web App (PWA) caching.

References

[1] Ministry of Electronics and Information Technology (MeitY), Government of India, "Digital Public Infrastructure for Citizen Empowerment," *NeGD Technical Report*, New Delhi, 2024.

[2] National e-Governance Division, "MyScheme Portal: One-Stop Search and Discovery of Government Schemes," *MeitY Whitepaper*, 2023.

[3] Direct Benefit Transfer (DBT) Mission, Cabinet Secretariat, Government of India, "DBT Performance Analytics and Beneficiary Dissemination," *Annual Report*, 2023.

[4] React Team, "React 19 Documentation: Server Components, Transitions, and Code-Splitting with Suspense," *Meta Open Source*, 2024.

[5] Supabase, "Row Level Security (RLS) and Multi-Tenant Architecture in PostgreSQL," *Supabase Documentation*, 2024.

[6] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," *Technical Report*, 2023.

[7] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.

[8] OWASP Foundation, "OWASP Top 10 API Security Risks: Broken Object Level Authorization (BOLA/IDOR)," *OWASP Technical Standard*, 2023.

[9] PostgreSQL Global Development Group, "pgvector: Open-Source Vector Similarity Search for PostgreSQL," *PostgreSQL Extensions Guide*, 2024.

[10] World Bank Group, "GovTech: Putting People First through Tech-Enabled Public Sector Solutions," *World Bank Working Paper No. 102*, Washington, D.C., 2022.